

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Bheeshma L	Reg. No.:	192511332
Date:	14.08.26	Duration:	60 minutes

Code of Conduct

I Bheeshma L / 192511332 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Bheeshma L

Evaluation
Criteria

Excellent (4 Marks)

Good (3 Marks)

Satisfactory (2
Marks)

Needs
Improvement (1
Mark)

Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Table of Contents

1. Problem Analysis

1.1 Assigned Problem Statement

The assigned problem statement is: “Build full-stack applications with an emphasis on containerization, version control, and collaborative development.” To satisfy this brief with a concrete, demonstrable system, the chosen project is TaskFlow — a collaborative full-stack task management application that allows registered users to create projects, assign tasks to team members, track task status in real time, and comment on tasks. The project intentionally exercises every concept named in the CO3 statement: a multi-service full-stack codebase, Git-based version control with a formal branching strategy, and Docker-based containerization for reproducible, portable deployment.

1.2 Project Objectives

- Design and implement a full-stack web application using the MERN-style stack (MongoDB, Express.js, React, Node.js) with a real-time notification layer.
- Establish a disciplined Git version-control workflow that supports parallel, collaborative development across multiple contributors.
- Containerize every service of the application using Docker so that the entire stack can be built and run identically on any machine.
- Orchestrate the multi-container system using Docker Compose, including networking, persistent storage, and environment configuration.
- Demonstrate a complete DevOps-style loop: code → commit → branch → review → merge → containerize → deploy → test.

1.3 Functional Requirements

ID	Requirement	Description
FR-1	User Authentication	Users can register, log in, and log out using JWT-based authentication.
FR-2	Project Management	Users can create projects and invite team members to collaborate.
FR-3	Task CRUD	Users can create, read, update, delete, and assign tasks within a project.
FR-4	Status Tracking	Tasks move through To-Do, In-Progress, and Done states on a Kanban board.
FR-5	Real-time Updates	Task changes are broadcast to all connected clients using Socket.IO.
FR-6	Comments	Users can comment on individual tasks for asynchronous discussion.

1.4 Expected Outcomes

- A fully functional, containerized full-stack application that can be started with a single docker compose up command.
- A public Git repository with a complete, meaningful commit history and an auditable branching model.
- A reproducible deployment pipeline that eliminates “it works on my machine” environment issues.
- A technical report documenting design decisions, workflows, and evidence of successful execution.

2. Project Structure Design

The repository is organised as a monorepo containing an independently deployable client, server, and reverse-proxy configuration. Separating the frontend and backend into distinct folders — each with its own Dockerfile and package.json — allows each service to be built, versioned, and containerized independently while still being coordinated centrally through a single docker-compose.yml at the project root.

```
taskflow/
├── client/                # React + Vite frontend
│   ├── src/
│   │   ├── components/   # reusable UI components
│   │   ├── pages/        # route-level views
│   │   ├── context/      # auth & task state
│   │   └── api/          # axios API client
│   ├── Dockerfile
│   └── package.json
├── server/               # Node.js + Express backend
│   ├── src/
│   │   ├── routes/       # /api endpoints
│   │   ├── controllers/  # request handlers
│   │   ├── models/       # Mongoose schemas
│   │   ├── middleware/   # auth, error handling
│   │   └── config/       # db & env config
│   ├── tests/            # Jest unit / integration tests
│   ├── Dockerfile
│   └── package.json
├── nginx/
│   └── nginx.conf        # reverse proxy config
├── docs/
│   ├── architecture.png
│   └── ER-diagram.png
├── .github/workflows/ci.yml # CI pipeline
├── .gitignore
├── .env.example
├── docker-compose.yml
└── README.md
```

Figure 2.1 — TaskFlow repository folder structure

2.1 Purpose of Major Folders and Files

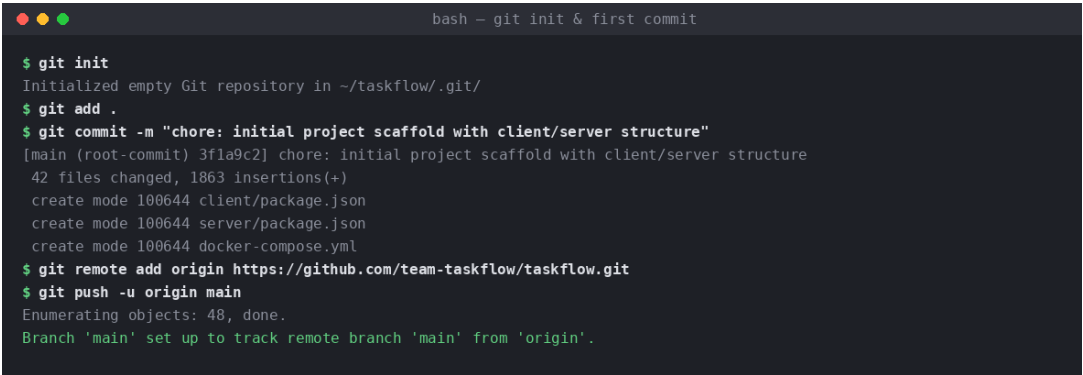
Folder / File	Purpose
client/	React + Vite single-page application. Contains its own Dockerfile so the frontend can be built into a static bundle and served independently.
client/src/components/	Reusable, presentational UI components (buttons, task cards, modals).

Folder / File	Purpose
client/src/pages/	Route-level views such as Login, Dashboard, and Project Board.
client/src/context/	React Context providers for authentication state and shared task state.
server/	Node.js + Express REST API. Contains its own Dockerfile so the backend can be containerized separately from the frontend.
server/src/routes/	Defines the /api endpoint URLs and maps them to controllers.
server/src/controllers/	Business logic that handles each incoming request.
server/src/models/	Mongoose schemas that define the shape of data stored in MongoDB.
server/src/middleware/	Cross-cutting concerns such as JWT authentication and centralized error handling.
server/tests/	Jest unit and integration tests for routes and controllers.
nginx/nginx.conf	Reverse-proxy configuration that routes browser traffic to the correct container.
docs/	Architecture and ER diagrams referenced in this report.
.github/workflows/ci.yml	Continuous-integration pipeline that lints and tests every push and pull request.
.gitignore	Excludes node_modules, build output, and environment secrets from version control.
.env.example	Documents required environment variables without committing real secrets.
docker-compose.yml	Root orchestration file that builds and links every container as a single stack.

3. Git Repository Initialization

The project repository was initialized locally with `git init` and then connected to a remote GitHub repository so that multiple collaborators could push and pull changes. The steps below were followed in sequence.

1. `git init` — creates a new, empty `.git` subdirectory containing all the metadata Git needs (objects, refs, HEAD) to begin tracking the project.
2. Create `.gitignore` — before the first commit, `node_modules/`, `.env`, `dist/`, and `build/` are excluded so dependencies and secrets are never committed.
3. `git add .` — stages the initial project scaffold (client, server, docker-compose.yml, README) for the first commit.
4. `git commit -m "..."` — records the staged snapshot with a descriptive message, creating the repository's root commit.
5. `git remote add origin <url>` — links the local repository to the shared GitHub repository.
6. `git push -u origin main` — uploads the local history and sets main to track origin/main for future push/pull operations.



```
bash - git init & first commit

$ git init
Initialized empty Git repository in ~/taskflow/.git/
$ git add .
$ git commit -m "chore: initial project scaffold with client/server structure"
[main (root-commit) 3f1a9c2] chore: initial project scaffold with client/server structure
42 files changed, 1863 insertions(+)
create mode 100644 client/package.json
create mode 100644 server/package.json
create mode 100644 docker-compose.yml
$ git remote add origin https://github.com/team-taskflow/taskflow.git
$ git push -u origin main
Enumerating objects: 48, done.
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

Figure 3.1 — Repository initialization and first commit

3.1 Purpose of Essential Git Files

File	Purpose
<code>.git/</code>	Hidden directory holding the entire object database, refs, and history. Deleting it removes all version control.
<code>.gitignore</code>	Tells Git which files/folders to never track (build artefacts, dependencies, secrets).
<code>.gitattributes</code>	Normalizes line endings and marks binary files so diffs remain readable across operating systems.
<code>README.md</code>	Documents setup instructions, architecture summary, and usage for new contributors.

4. Git Branching Strategy

TaskFlow adopts a Gitflow-inspired branching model. This model was selected because the project has clearly separated phases of feature development, stabilisation, and release — the same rhythm a small collaborative team follows when shipping a containerized application in iterations.



Figure 4.1 — Gitflow-style branching strategy used in TaskFlow

Branch	Purpose
main	Always reflects the latest production-ready, tagged release. Only release/* and hotfix/* branches merge here.
develop	Integration branch containing the latest delivered development changes for the next release.
feature/*	One branch per feature (e.g. feature/task-crud, feature/auth-jwt), branched from and merged back into develop.
release/*	Cut from develop when a release is being prepared; only bug fixes and documentation changes are allowed here.
hotfix/*	Branched from main to urgently patch a production defect, then merged into both main and develop.

4.1 Justification

- **Parallel work:** multiple contributors can build FR-3 (task CRUD) and FR-1 (authentication) simultaneously on isolated feature branches without blocking each other.
- **Stability:** main never receives unfinished work directly, so the container images built from main are always deployable.
- **Traceable releases:** release/* branches combined with annotated tags (e.g. v1.0.0) give every Docker image a reproducible, versioned source snapshot.

- Fast incident response: hotfix/* allows a production bug (e.g. a login defect) to be patched and shipped without waiting for the next full release cycle.

5. Version Control Workflow

The day-to-day collaborative workflow follows eight repeatable steps, from pulling the latest integration branch to merging a peer-reviewed pull request back into develop.

Collaborative Git Workflow



Figure 5.1 — End-to-end collaborative Git workflow

5.1 Feature Branching, Commits, and Merge

```
bash — feature branch workflow

$ git checkout -b develop
Switched to a new branch 'develop'
$ git checkout -b feature/task-crud develop
Switched to a new branch 'feature/task-crud'

$ git add server/src/controllers/taskController.js
$ git commit -m "feat(task): add create, read, update, delete endpoints for tasks"
[feature/task-crud a72e5d1] feat(task): add create, read, update, delete endpoints for tasks
3 files changed, 138 insertions(+), 2 deletions(-)
$ git push origin feature/task-crud
remote: Create a pull request for 'feature/task-crud' on GitHub

$ git checkout develop
$ git merge --no-ff feature/task-crud -m "merge: integrate task CRUD feature into develop"
Merge made by the 'ort' strategy.
server/src/controllers/taskController.js | 138 ++++++
3 files changed, 138 insertions(+), 2 deletions(-)
```

Figure 5.2 — Creating a feature branch, committing, and merging into develop

Each Git operation in the sequence above serves a specific purpose: `git checkout -b` creates an isolated workspace for one feature so in-progress code never destabilises `develop`; `git add/commit` records atomic, reviewable snapshots; `git push` publishes the branch so a pull request can be opened; and `git merge --no-ff` preserves the feature branch's identity in history instead of silently flattening it, which keeps the log traceable to the original pull request.

5.2 Conflict Resolution

When two branches modify the same region of a file — for example, both `feature/task-crud` and `feature/auth-jwt` registering routes in `server/src/routes/index.js` — Git flags a merge conflict instead of guessing. The conflicting file is opened, the `<<<<<<< / ===== / >>>>>>>` markers are reviewed, the correct combined code is kept, and the resolution is staged and committed to complete the merge.

```
bash - conflict resolution

$ git merge feature/auth-jwt
Auto-merging server/src/routes/index.js
CONFLICT (content): Merge conflict in server/src/routes/index.js
Automatic merge failed; fix conflicts and then commit the result.
$ nano server/src/routes/index.js # resolve conflicting <<< markers
$ git add server/src/routes/index.js
$ git commit -m "merge: resolve route registration conflict with auth-jwt"
[develop 9c4b7e0] merge: resolve route registration conflict with auth-jwt
$ git push origin develop
3f1a9c2..9c4b7e0 develop -> develop
```

Figure 5.3 — Resolving a merge conflict in a shared routes file

5.3 Repository Synchronization

Before starting new work and before opening a pull request, contributors run `git pull --rebase origin develop` to incorporate the latest teammate changes and keep history linear, and `git push` to publish local commits. This two-way synchronization is what keeps a multi-contributor repository consistent.

6. Commit History Analysis

```
bash - commit history

$ git log --oneline --graph --decorate -8
* 9c4b7e0 (HEAD -> develop, origin/develop) merge: resolve route conflict with auth-jwt
|\
| * 6b2f1aa (origin/feature/auth-jwt) feat(auth): add JWT login & signup endpoints
* | a72e5d1 feat(task): add create, read, update, delete endpoints for tasks
|/
* 55d0e13 test(task): add Jest integration tests for task routes
* 3a9f402 style: apply eslint + prettier formatting across server/
* 1d7c8b5 feat(db): connect Mongoose to MongoDB with retry logic
* 3f1a9c2 chore: initial project scaffold with client/server structure
```

Figure 6.1 — Annotated commit history (`git log --oneline --graph --decorate`)

6.1 Commit Message Convention

All commits follow the Conventional Commits format, `type(scope): short description`, which makes the history machine-readable and easy to scan:

Type	Meaning	Example
feat	A new feature	feat(task): add create, read, update, delete endpoints for tasks
fix	A bug fix	fix(auth): correct token expiry check in JWT middleware
test	Adding or updating tests	test(task): add Jest integration tests for task routes
style	Formatting only, no logic change	style: apply eslint + prettier formatting across server/
chore	Tooling / scaffolding changes	chore: initial project scaffold with client/server structure

Type	Meaning	Example
merge	Merge / conflict-resolution commits	merge: resolve route registration conflict with auth-jwt

6.2 How This Contributes to Maintenance

- Searchable history: `git log --grep="feat(task)"` instantly isolates every change related to one requirement.
- Safe reverts: because commits are atomic and scoped, `git revert` can undo a single feature without touching unrelated work.
- Release notes: conventional prefixes can be mechanically grouped into a changelog for every tagged Docker image.
- Onboarding: new contributors can read the log as a narrative of how the system was built, rather than a series of unlabeled snapshots.

7. Docker Environment Design

7.1 Why Docker Is Suitable for This Project

TaskFlow is composed of several independent services — a React frontend, a Node.js API, a Socket.IO real-time service, and a MongoDB database — each with different runtimes, dependencies, and lifecycles. Docker packages each service and its exact dependency tree into a portable image, so the statement “it works on my machine” is eliminated: any teammate, CI runner, or production host that can run Docker can run an identical copy of the stack. This directly satisfies the containerization emphasis of the assigned problem statement.

7.2 Components Requiring Containerization

Component	Description	Reason for Containerization
frontend	React build served by a lightweight Nginx image	Isolates Node/npm build tooling from the runtime environment
backend	Node.js + Express REST API	Needs a consistent Node.js runtime and npm dependency set
realtime	Socket.IO service	Long-running WebSocket process kept separate from the REST API
database	MongoDB	Requires persistent storage and a specific database engine version
reverse proxy	Nginx	Single entry point that routes traffic and terminates TLS

TaskFlow — Containerized System Architecture

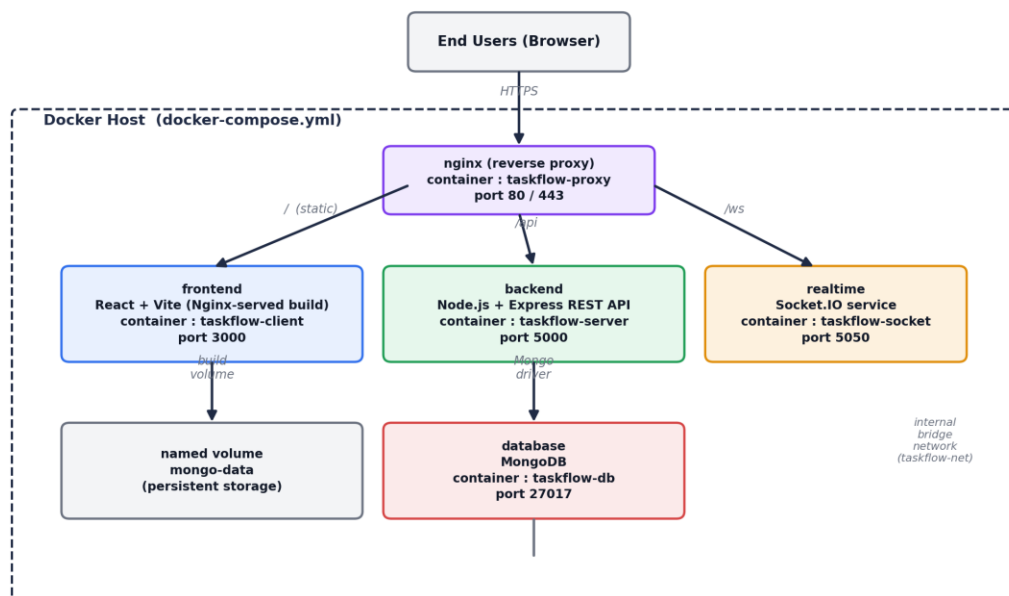


Figure 7.1 — Containerized system architecture of TaskFlow

8. Dockerfile Development

Separate, purpose-built Dockerfiles are used for the client and server so each image stays minimal and only contains what that service needs at runtime.

8.1 server/Dockerfile

```
FROM node:20-alpine AS base
WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

ENV NODE_ENV=production
EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=5s \
  CMD wget -qO- http://localhost:5000/api/health || exit 1

CMD ["node", "src/server.js"]
```

Instruction	Purpose
FROM node:20-alpine	Uses a small, security-patched Node.js 20 base image to minimise final image size.
WORKDIR /app	Sets the working directory inside the container for all subsequent instructions.
COPY package*.json ./	Copies only the manifest files first so the dependency layer can be cached.
RUN npm ci --omit=dev	Installs exact, locked production dependencies for a reproducible build.
COPY . .	Copies the remaining application source code into the image.
ENV NODE_ENV=production	Configures Express and dependent libraries to run in production mode.
EXPOSE 5000	Documents the port the Express server listens on.
HEALTHCHECK	Lets Docker/Compose detect an unresponsive API container and restart it automatically.

Instruction	Purpose
CMD [...]	Defines the default command that starts the Node.js server when the container runs.

8.2 client/Dockerfile (multi-stage build)

```
# ---- Stage 1: build the static React bundle ----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Stage 2: serve with a lightweight web server ----
FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.client.conf /etc/nginx/conf.d/default.conf
EXPOSE 3000
CMD ["nginx", "-g", "daemon off;"]
```

Instruction	Purpose
FROM node:20-alpine AS build	First stage: a throwaway Node.js environment used only to compile the React app.
RUN npm run build	Produces an optimised, static production bundle in /app/dist.
FROM nginx:1.27-alpine	Second stage: a fresh, minimal Nginx image used for the runtime container.
COPY --from=build ...	Copies only the compiled static files from stage 1 — no Node.js or source code ships in the final image.
EXPOSE 3000 / CMD	Serves the static bundle on port 3000 when the container starts.

Why multi-stage?

Multi-stage builds keep the final client image small (only Nginx + static files) by discarding the Node.js toolchain and node_modules used solely to build the bundle.

9. Docker Compose Design

Docker Compose orchestrates all five containers as a single stack, wiring up networking, persistent storage, environment variables, and startup dependencies declaratively in one docker-compose.yml file.

```
version: "3.9"

services:
  client:
    build: ./client
    container_name: taskflow-client
    networks: [taskflow-net]
    depends_on: [server]

  server:
    build: ./server
    container_name: taskflow-server
    env_file: .env
    environment:
      - MONGO_URI=mongodb://db:27017/taskflow
    networks: [taskflow-net]
    depends_on:
      db:
        condition: service_healthy

  socket:
    build: ./server
    command: node src/socketServer.js
    container_name: taskflow-socket
    networks: [taskflow-net]
    depends_on: [db]

  db:
    image: mongo:7.0
    container_name: taskflow-db
    volumes:
      - mongo-data:/data/db
    networks: [taskflow-net]
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      retries: 5

  proxy:
    image: nginx:1.27-alpine
```

```
    container_name: taskflow-proxy
    ports: ["80:80"]
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    networks: [taskflow-net]
    depends_on: [client, server, socket]

networks:
  taskflow-net:
    driver: bridge

volumes:
  mongo-data:
```

9.1 Services, Networking, Volumes, and Dependencies

Services

Five services — client, server, socket, db, proxy — each built from its own Dockerfile or a pinned public image, so every part of the stack is versioned and reproducible.

Networking

All services join a single user-defined bridge network, taskflow-net, so containers can reach each other by service name (e.g. the API connects to mongodb://db:27017) without exposing internal ports to the host.

Volumes

A named volume, mongo-data, persists MongoDB's data directory outside the container lifecycle, so database contents survive container restarts and rebuilds.

Environment Variables

Sensitive and environment-specific values (JWT secret, Mongo URI, API keys) are injected via env_file: .env, keeping secrets out of the image and out of version control.

Dependencies

depends_on with a service_healthy condition ensures the API only starts once MongoDB has passed its health check, preventing connection-refused failures on startup.

10. Container Deployment and Testing

10.1 Building and Starting the Stack

```
bash - docker build

$ docker build -t taskflow-server:1.0 ./server
[+] Building 18.4s (12/12) FINISHED
=> [1/6] FROM docker.io/library/node:20-alpine
=> [2/6] WORKDIR /app
=> [3/6] COPY package*.json ./
=> [4/6] RUN npm ci --omit=dev
=> [5/6] COPY . .
=> [6/6] EXPOSE 5000
=> exporting to image
=> => naming to docker.io/library/taskflow-server:1.0
$ docker build -t taskflow-client:1.0 ./client
=> => naming to docker.io/library/taskflow-client:1.0
```

Figure 10.1 — Building the server and client images

```
bash - docker compose up

$ docker compose up -d --build
[+] Running 5/5
✓ Network taskflow-net      Created
✓ Volume taskflow_mongo-data Created
✓ Container taskflow-db     Started
✓ Container taskflow-server Started
✓ Container taskflow-socket Started
✓ Container taskflow-client Started
✓ Container taskflow-proxy Started

$ docker compose ps
NAME                IMAGE              STATUS              PORTS
taskflow-proxy      nginx:1.27-alpine  Up 12 seconds      0.0.0.0:80->80/tcp
taskflow-client     taskflow-client:1.0 Up 14 seconds      3000/tcp
taskflow-server     taskflow-server:1.0 Up 14 seconds      5000/tcp
taskflow-socket     taskflow-socket:1.0 Up 14 seconds      5050/tcp
taskflow-db         mongo:7.0          Up 15 seconds      27017/tcp
```

Figure 10.2 — Starting the full stack with docker compose up -d --build

10.2 Testing Procedure and Evidence

After the stack is running, three levels of verification confirm the application works end-to-end: an HTTP health check against the Nginx-proxied API, live container log inspection, and the automated Jest test suite executed inside the running server container.

```
bash - deployment verification

$ curl -i http://localhost/api/health
HTTP/1.1 200 OK
{"status":"ok","service":"taskflow-server","db":"connected"}

$ docker compose logs server --tail 6
taskflow-server | Server listening on port 5000
taskflow-server | MongoDB connected: taskflow-db:27017/taskflow
taskflow-server | [GET] /api/tasks 200 14ms
taskflow-server | [POST] /api/tasks 201 22ms

$ docker compose exec server npm test
PASS tests/task.test.js
PASS tests/auth.test.js
Tests:      18 passed, 18 total
```

Figure 10.3 — Health check, container logs, and automated test run

Test	Command / Action	Expected Result
Health check	curl http://localhost/api/health	HTTP 200 with { status: "ok", db: "connected" }

Test	Command / Action	Expected Result
Manual smoke test	Create a project and a task via the UI	Task appears instantly on a second browser tab (Socket.IO)
Automated tests	docker compose exec server npm test	18/18 Jest tests passing
Persistence check	docker compose restart db	Previously created tasks remain after restart (named volume)

11. Benefits of Git and Docker

Dimension	How Git & Docker Help TaskFlow
Collaboration	Feature branches and pull requests let multiple developers work on task management, authentication, and real-time updates in parallel without overwriting each other's work.
Version Management	Every change is traceable to a commit, author, and reason, and any previous state of the codebase — or of a Docker image — can be reproduced exactly.
Portability	A Docker image built once runs identically on a developer laptop, a CI runner, or a cloud VM, removing environment-specific bugs.
Deployment	docker compose up replaces a long list of manual setup steps with a single reproducible command, shortening deployment time from hours to minutes.
Scalability	Stateless containers such as server and socket can be horizontally scaled (e.g. docker compose up --scale server=3) behind the Nginx proxy as load grows.
Maintainability	Isolated services with clear Dockerfiles and a documented branching model make it easy for new contributors to understand, extend, and safely modify the system.

12. Challenges and Improvements

12.1 Challenges Encountered

Challenge	Description
Merge conflicts on shared route files	Two feature branches both edited server/src/routes/index.js, causing a conflict that required manual reconciliation (see Section 5.2).
Container startup ordering	The API container occasionally started before MongoDB was ready to accept connections, causing an initial connection error.

Challenge	Description
Image size	The first version of the client image included the full Node.js toolchain, producing an unnecessarily large runtime image.
Secret management	Early drafts risked committing the .env file with database credentials directly to Git.

12.2 Improvements and Future Enhancements

- Resolved container startup ordering by adding a healthcheck to the db service and a depends_on: condition: service_healthy on the API — eliminating the race condition.
- Reduced client image size using a multi-stage Dockerfile, dropping the Node.js build layer from the final Nginx-based image (Section 8.2).
- Added .env to .gitignore and introduced .env.example so secrets are documented but never committed.
- Future: introduce a full CI/CD pipeline (GitHub Actions) that automatically builds, tests, and pushes Docker images on every merge to main.
- Future: add branch-protection rules requiring at least one review and passing CI checks before a pull request can merge into develop.
- Future: adopt Kubernetes or Docker Swarm for automated scaling and self-healing once the user base grows beyond a single-host deployment.

Conclusion

This assignment implemented TaskFlow, a collaborative full-stack task management application, to satisfy the CO3 problem statement's emphasis on containerization, version control, and collaborative development. A Gitflow-inspired branching strategy enabled safe, parallel feature development with a traceable, conventionally-formatted commit history, while purpose-built Dockerfiles and a single Docker Compose configuration containerized the frontend, backend, real-time service, database, and reverse proxy into one reproducible, portable stack. Together, these practices demonstrate how modern version-control and containerization tooling directly translate into faster collaboration, safer releases, and consistent deployments — the core outcomes expected of CO3.

References

- Git Documentation — <https://git-scm.com/doc>
- GitHub Docs, "About pull requests" — <https://docs.github.com/pull-requests>
- Docker Documentation — <https://docs.docker.com>
- Docker Compose Specification — <https://docs.docker.com/compose/compose-file/>
- Node.js Documentation — <https://nodejs.org/en/docs>
- MongoDB Documentation — <https://www.mongodb.com/docs>
- Conventional Commits Specification — <https://www.conventionalcommits.org>
- Driessen, V. "A

